Chapter 4: Pandas: Reading/Writing

<{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to read data from CSV, Excel, JSON, and SQLite files using appropriate Pandas functions and parameters
<{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to construct correct relative and absolute file paths for local files and mounted cloud storage environments
<{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to write DataFrames to multiple output formats including CSV, Excel, JSON, and SQLite with appropriate options
<{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to create and query SQLite databases, writing DataFrames directly using .to_sql() and retrieving results with pd.read_sql()

4.1Introduction

Reading and writing data (a.k.a. file input/output [IO]) is one of the key skills for any data work with Python. This chapter covers how to read from and write to:

- CSV files

- SQL databases
- JSON objects

4.2Reading Data from File

The techniques for accessing files are slightly different for some IDEs. You are welcome to learn how each IDE accesses files, but you only need to watch one of the videos below.

This video can be viewed online.

Accessing Files with Google Colab

This video can be viewed online.

Accessing Files with Anaconda/Jupyter Notebook

The Pandas library package makes reading and writing data from files an easy task. Begin by importing the Pandas package, then locate the data file on your machine:

```
import pandas as pd

df = pd.read_csv('file_name.csv')
```

For Excel, we use a similar command:

```
import pandas as pd

df = pd.read_excel('file_name.xlsx', 'Sheet_name', index_col = None, na_values=
['NA'])
```

The above commands only read the data from the files and give you a sneak peek of what the data looks like. If you want to read the data and manipulate it, you will need to save the output to a variable.

```
df = pd.read_csv('file_name.csv')
df.head() # This will give you a preview of the first five lines of your dataset
```

This assigns the output of the file_name.csv to the 'df' variable. Why did I name it 'df'? As you might have guessed, the output of the read_csv() method is a pandas DataFrame object. That means the read_csv() constructor has its own parameter options to match most of the DataFrame parameters (data, index, columns, and dtype) plus many more that apply to CSV files. But before getting into those details, it helps to understand how to locate the CSV file in a variety of contexts.

Absolute vs. Relative Paths

In the examples above, we used relative paths to find the CSV files. You can use either relative or absolute paths in Python:

- A relative path The path to a file if you start from your current working directory. is the path to the file if you start from your current working directory. You can drill down into the folders in your working directory by using the "/" character and ".." to access the parent folder (i.e., directory). Essentially, you are working your way up and down a tree structure until you find the file you are looking for. See below:
    - Drill down into child folders:

        - 'datafolder/subfolder/file_name.csv'
    - Move up into parent folders:
        - # up one folder

          '../file_name.csv'

          # two levels up

          '../../file_name.csv'

          # two levels up and then down another child branch

          '../../differentfolder/file_name.csv'

    You will often need to use relative paths when you access files on a server or mounted drive (e.g., like we do with Colab and the mounted Google Drive).

- An `absolute path` The complete path from the base of your file system to the file that you want to load. is the complete path from the base of your file system to the file that you want to load.
    - # read a CSV file from a location on the main "C" drive

      'c:/foldername/subfoldername/file_name.csv'

      # you can also read from additional drives attached to the same computer, like an external hard drive, USB drive, second internal hard drive, or partition.

      'd:/foldername/subfoldername/file_name.csv'

    You can use absolute paths any time that your data resides on your own computer. You may also be able to use an absolute path when working with files on a server, but it depends on the server's permission levels. The video at the beginning of this chapter that introduces the Jupyter Notebook example on reading CSV files uses an absolute path.

Practice with CSV

Try it yourself. Download the CSV file below. It's a freely available dataset from Kaggle datasets. Each record represents a medical insurance customer and several pieces of information, including sex, BMI, age, region, smoker, gender, and charges. Charges is a measure of how much that customer has cost the insurance company.

This download can be found online.

If you are using **Jupyter Notebook** (or some other IDE), then select an organized location on your computer to keep all of the data files you will access from Python. The first example below demonstrates how to access insurance.csv from the same folder you are using for a Jupyter Notebook workspace:

```
df = pd.read_csv('insurance.csv')
df.head()
```

In this next example, you are accessing insurance.csv from a child folder called 'data', which is a child of 'class', which is in your Jupyter Notebook workspace:

```
df = pd.read_csv('class\data\insurance.csv')
df.head()
```

In this next example, you are accessing insurance.csv using an absolute path. Notice that even the drive letter ('D:\') is different from the drive the Jupyter Notebook workspace actually resides in ('C:\'). In other words, although you can't navigate to a parent folder using a relative path, you can use an absolute path to get to any location on your computer:

```
df = pd.read_csv('D:\Google Drive\Colab Notebooks\data\insurance.csv')
df.head()
```

If you are using **Google Colab**, then you first need to mount a virtual drive to your Google Drive folder (the cloud version, not the version on your computer if you are using Backup and Sync). Follow the process after executing the code below:

```
from google.colab import drive

drive.mount('/content/drive')
```

Once you have entered your code and Google Drive is mounted, you can read a file using a relative path. In the example below, the insurance.csv file is inside a folder called 'data' inside the 'Google Colab' folder:

```
df = pd.read_csv('/content/drive/My Drive/Colab Notebooks/data/insurance.csv')
df.head()
```

	age	sex	bmi	children	smoker	region	charges
0	19	female	27.900	0	yes	southwest	16884.92400
1	18	male	33.770	1	no	southeast	1725.55230
2	28	male	33.000	3	no	southeast	4449.46200
3	33	male	22.705	0	no	northwest	21984.47061
4	32	male	28.880	0	no	northwest	3866.85520

Read CSV from URL

It is worth noting that you can also read CSV files online that are available to download directly from websites. The process is much simpler in Google Colab because Google Colab doesn't require that you mount a virtual drive path to your Google Drive directory.

Throughout the remainder of this book, the examples provided may use either a local file or a URL. We will typically provide you with the file for download so that you can use it locally even if the example pulls it from a URL. We recommend you always download the file provided and read it locally because that will be faster and more reliable than reading from a URL which can occasionally be unavailable.

```
# Read CSV from a URL
import pandas as pd

df = pd.read_csv('/content/drive/My Drive/Colab Notebooks/data/insurance.csv')
df.head()
```

	age	sex	bmi	children	smoker	region	charges
0	19	female	27.900	0	yes	southwest	16884.92400
1	18	male	33.770	1	no	southeast	1725.55230
2	28	male	33.000	3	no	southeast	4449.46200
3	33	male	22.705	0	no	northwest	21984.47061
4	32	male	28.880	0	no	northwest	3866.85520

Read_csv() Constructor

As mentioned above, the read_csv() constructor has most of the same parameters as DataFrame() (since they both output a DataFrame). However,

there are several parameters that allow you to adjust elements, such as the character encoding of the CSV file and the delimiter of the values in the file (comma, tab, space, etc.). For now, simply review the sample of the more common parameters below:

Read_csv()
See the documentation here.

Default values: *pandas.read_csv(filepath_or_buffer: Union[str, pathlib.Path, IO[~AnyStr]], sep=',', delimiter=None, header='infer', names=None, index_col=None, usecols=None, squeeze=False, prefix=None, mangle_dupe_cols=True, dtype=None, engine=None, converters=None, true_values=None, false_values=None, skipinitialspace=False, skiprows=None, skipfooter=0, nrows=None, na_values=None, keep_default_na=True, na_filter=True, verbose=False, skip_blank_lines=True, parse_dates=False, infer_datetime_format=False, keep_date_col=False, date_parser=None, dayfirst=False, cache_dates=True, iterator=False, chunksize=None, compression='infer', thousands=None, decimal=b'.', lineterminator=None, quotechar='"', quoting=0, doublequote=True, escapechar=None, comment=None, encoding=None, dialect=None, error_bad_lines=True, warn_bad_lines=True, delim_whitespace=False, low_memory=True, memory_map=False, float_precision=None)*

- *filepath_or_buffer*: as demonstrated previously, this is a file path (whether relative or absolute) or URL to the CSV file
- *delimiter (alias for 'sep'; both do the same thing)*: this is the character(s) that separates values in the CSV file; it will usually be a comma, but sometimes other characters are also used, like a tab (delimiter='\t')

- *header*: this is the line that the column names begin on; it defaults to the first row; useful when there are extra lines above the header you want to ignore
- *skiprows*: this is a list of rows to skip; useful when there are extra rows between your header and the data you want (e.g., Qualtrics CSV data files do this)
- *dtype*: specifies a datatype for all columns; defaults to selecting the smallest data type possible for each column (similar to the DataFrame constructor)
- *quotechar*: the character used to denote the start and end of a quoted item; quoted items can include the delimiter and it will be ignored
- *comment*: indicates remainder of line should not be parsed; if found at the beginning of a line, the line will be ignored altogether; this parameter must be a single character; like empty lines (as long as skip_blank_lines=True), fully commented lines are ignored by the parameter header but not by skiprows; for example, if comment='#', parsing #empty\na,b,c\n1,2,3 with header=0 will result in ‘a,b,c’ being treated as the header
- *encoding*: specifies which type of character encoding to assume when reading data; often text data comes with any character that can be inputted by the user’s keyboard; some of these characters will generate a runtime error; specifying a larger-ranged character encoding, such as 'UTF-16' or 'ISO-8859-1', will often fix this
- *low_memory*: imports massive data files; it means that Python will process the file in chunks, resulting in lower memory use while parsing but possibly creating a mixed type inference; To ensure no mixed types, either set False or specify the type with the dtype parameter; note that the

entire file is read into a single DataFrame regardless, so use the chunksize or iterator parameter to return the data in chunks

Notice what happens when you try to print a DataFrame (or any other object) with many rows and columns that could take up an enormous amount of screen space. Only the first *n* and last *n* rows (and columns if we have too many of them) are shown, with "..." indicating that there are more hidden in the middle.

```
df
```

	age	sex	bmi	children	smoker	region	charges
0	19	female	27.900	0	yes	southwest	16884.92400
1	18	male	33.770	1	no	southeast	1725.55230
2	28	male	33.000	3	no	southeast	4449.46200
3	33	male	22.705	0	no	northwest	21984.47061
4	32	male	28.880	0	no	northwest	3866.85520
...	...	...	...	...	...	...	...
1333	50	male	30.970	3	no	northwest	10600.54830
1334	18	female	31.920	0	no	northeast	2205.98080
1335	18	female	36.850	0	no	southeast	1629.83350
1336	21	female	25.800	0	no	southwest	2007.94500
1337	61	female	29.070	0	yes	northwest	29141.36030

1338 rows × 7 columns

If you want to see the entire set of rows (or columns), use the .set_option() methods below:

```
import pandas as pd # This line is only necessary if you don't already have pandas in memory from a prior code block

df = pd.read_csv('/content/drive/My Drive/Colab Notebooks/data/insurance.csv')
pd.set_option('display.max_rows', 1000000) # You can adjust the number to anything you want
pd.set_option('display.max_columns', 1000) # You can adjust the number to anything you want
df
```

For space concerns, I won't actually display the output here, but you get the idea.

4.3Writing Data to a File

This video can be viewed online.

After reading and manipulating your data, you will often need to write the results into a file or database. Writing to a CSV file is straightforward:

```
# Assuming we have written our data to a variable called cleaned_data
cleaned_data.to_csv('file_to_write_to.csv')
```

For an Excel file:

```
cleaned_data.to_excel('name of Excel file you want to write to.xlsx', sheet_name = 'sheet1')
```

You can also insert records into a specific position of a dataset using .loc[]. Pass the position as an index, then set it equal to the value you want:

```
data = pd.read_csv('some_file.csv')
data.loc[0] = 'This is the item I want to insert'
```

The next examples demonstrate writing to an actual file by looping through a list, dictionary, and DataFrame. This is also a good time to learn a few new methods associated with collections that are useful when iterating through multiple lists/columns at once. The most basic example loops through a list and writes it to a file:

```
import pandas as pd

# NOTE: if you are using Google Colab, make sure you have already mounted Google
Drive:
# from google.colab import drive
# drive.mount('/content/drive')
day = 0 # We initialize this variable at 0 because we always start at 0 when dealing
with indexes
temperature = [55,61,72,75,63,59,43] # This is the temperature of every day last
week

# Create an empty DataFrame to store values in. Provide column name
df = pd.DataFrame(columns=['Temperature'])

for temp in temperature:
  df.loc[day] = temp
  day += 1 # We want to increment this so that the next time we go through this
loop,
             # we put the temperature in the next row
# Write to file
df.to_csv('/content/drive/My Drive/Colab Notebooks/data/weather.csv')

# Let's see what that new file looks like:
new_csv = pd.read_csv('/content/drive/My Drive/Colab Notebooks/data/weather.csv')
new_csv.head(5)
```

Output:

	Unnamed: 0	Temperature
0	0	55
1	1	61
2	2	72
3	3	75
4	4	63

This next example loops through a dictionary, adds a date for the temperature, and uses that date as the index. Notice that it does not allow you to use the index number. You can’t use the .iloc method. Rather, you must have a list of index values (i.e., 'date' in this context)

```
import pandas as pd

# Because we will use date as the index, we don't make it a named column in the
DataFrame
df = pd.DataFrame(columns=['Temperature'])

# Dictionary of date/temperature pairs
weather_dict = {'03-21-2021':51, '03-22-2021':61, '03-23-2021':72, '03-24-2021':75,
'03-25-2021':63, '03-26-2021':59, '03-27-2021':43}

# Use key, value pair to iterate through the dictionary: key=date, value=temp
for date, temp in weather_dict.items():
  df.loc[date] = temp

df.to_csv('/content/drive/My Drive/Colab Notebooks/data/weather1.csv')
new_csv = pd.read_csv('/content/drive/My Drive/Colab Notebooks/data/weather1.csv')
new_csv
```

Output:

	Unnamed: 0	Temperature
0	03-21-2021	51
1	03-22-2021	61
2	03-23-2021	72
3	03-24-2021	75
4	03-25-2021	63

The `zip()` Python function that binds two or more iterables together as tuples so that the index of each list matches up to represent common attributes of a single case. function, which binds two or more iterables together as tuples so that the index of each list matches up to represent common attributes of a single case. For example, index 0 refers to a temperature of 51 degrees on March 21, 2021. Note that if you zip with different-sized lists/columns, the resulting tuple will end at the last index where both lists have data.

```
import pandas as pd

df = pd.DataFrame(columns=['Temperature'])

# Separate lists for dates and high temperatures
dates = ['03-21-2021', '03-22-2021', '03-23-2021', '03-24-2021', '03-25-2021', '03-
26-2021', '03-27-2021']
highs = [55, 61, 72, 75, 63, 59, 43]

for date, temp in zip(dates, highs):
  df.loc[date] = (temp)

# Give the index a nice name and then write to CSV
df.index.name = 'Date'
df.to_csv('/content/drive/My Drive/Colab Notebooks/data/weather2.csv')
new_csv = pd.read_csv('/content/drive/My Drive/Colab Notebooks/data/weather2.csv')
new_csv.head(5)
```

Output:

	Date	Temperature
0	03-21-2021	55
1	03-22-2021	61
2	03-23-2021	72
3	03-24-2021	75
4	03-25-2021	63

Finally, the `enumerate()` Python function that adds a counter to an iterable object (e.g., list, dictionary, DataFrame) so that you can keep track of the index. function allows us to add a counter to an iterable object (e.g., list, dictionary, DataFrame) so that you can keep track of the index. It's particularly useful when looping through one list while referring to the associated index of another list(s). This technique is most useful when (1) you want to store all of your lists without using one of them as an index and (2) you want to loop through one list and also refer to the corresponding values of one or more separate lists or columns by numeric index.

```
import pandas as pd

df = pd.DataFrame(columns=['Date', 'Low temp', 'High temp'])

# Separate lists for dates and temperatures
dates = ['03-21-2021', '03-22-2021', '03-23-2021', '03-24-2021', '03-25-2021', '03-
26-2021', '03-27-2021']
highs = [55, 61, 72, 75, 63, 59, 43]
lows = [29, 31, 38, 40, 35, 33, 28]
```

```
for i, date in enumerate(dates):
  # Use i to track the index of the other lists not included in the 'for'
  # statement (lows and highs). Use the variable 'date' created in the 'for'
  # statement to refer to the date. Append each loop to the new DataFrame
  # and ignore the index so that it doesn't show up in the CSV file
  df2 = pd.DataFrame([[date, lows[i], highs[i]]], columns=['Date', 'Low temp', 'High
temp'])
  df = pd.concat([df, df2], ignore_index=True)

# Write to file without storing the index
df.to_csv('/content/drive/My Drive/Colab Notebooks/data/weather3.csv', index=False)
new_csv = pd.read_csv('/content/drive/My Drive/Colab Notebooks/data/weather3.csv')
new_csv
```

Output:

	Date	Low temp	High temp
0	03-21-2021	29	55
1	03-22-2021	31	61
2	03-23-2021	38	72
3	03-24-2021	40	75
4	03-25-2021	35	63
5	03-26-2021	33	59
6	03-27-2021	28	43

4.4JSON

This video can be viewed online.

JSON is the most popular format for sending and receiving data through REST web services and other contexts. Thankfully, there are native Python methods for both converting collection objects into JSON (and vice versa) as well as some useful DataFrame methods in Pandas for reading from and to JSON files and objects.

The most common use case involves writing to (.to_json Pandas method to write JSON formatted data.; see documentation here) and reading from (.read_json() Pandas method to read JSON formatted data.; see documentation here) between JSON objects and Pandas DataFrames. JSON interactions come up again later in the context of machine learning deployment. The JSON package A Python package used to convert strings into JSON dictionaries. is also used here to convert .read_json() objects into JSON dictionaries.

DataFrames can be read into six orientations of JSON formats. As you go through the examples below, it may be useful to remember that list items are referenced by an index number (e.g., [0]) whereas dictionary items are referenced by key name (e.g., ['age']). The default ('column') format looks like this:

```
import pandas as pd
import json

df = pd.read_csv('/content/drive/My Drive/Colab Notebooks/data/insurance.csv')
df = df.head(3)
json_str = df.to_json() # Default = 'columns'

print(f'df.to_json() resulting data type: {type(json_str)}')
df_json = json.loads(json_str)
print(f'json.loads() resulting data type: {type(df_json)}\n')
print(f'Columns (default) orientation:\n{df_json}')
```

```
# How to drill down: dictionary of dictionaries
print(df_json['age'])
print(df_json['age']['0'])
print(df_json['bmi']['1'])

# Read json back into DataFrame using same format
df_new = pd.DataFrame(df_json)


# Output:
# df.to_json() resulting data type: &lt;class 'str'&gt;
# json.loads() resulting data type: &lt;class 'dict'&gt;
# Columns (default) orientation:
# {'age': {'0': 19, '1': 18, '2': 28}, 'sex': {'0': 'female', '1': 'male', '2':
'male'}, 'bmi': {'0': 27.9, '1': 33.77, '2': 33.0}, 'children': {'0': 0, '1': 1, '2': 3},
'smoker': {'0': 'yes', '1': 'no', '2': 'no'}, 'region': {'0': 'southwest', '1':
'southeast', '2': 'southeast'}, 'charges': {'0': 16884.924, '1': 1725.5523, '2':
4449.462}}
# {'0': 19, '1': 18, '2': 28}
# 19
```

The Split orientation separates columns, index, and data into distinct keys:

```
json_str = df.to_json(orient='split')
df_json = json.loads(json_str)

print(f'Split orientation:\n{df_json}')

# How to drill down: dictionary of lists
print(df_json['columns'])
print(df_json['columns'][0])
print(df_json['data'])
print(df_json['data'][0])
print(df_json['data'][0][0])
print(df_json['data'][1][2])

# Read json back into DataFrame using same format
df_new = pd.DataFrame(pd.read_json(json_str, orient=&quot;split&quot;))


# Output:
# Split orientation:
# {'columns': ['age', 'sex', 'bmi', 'children', 'smoker', 'region', 'charges'],
'index': [0, 1, 2], 'data': [[19, 'female', 27.9, 0, 'yes', 'southwest', 16884.924], [18,
'male', 33.77, 1, 'no', 'southeast', 1725.5523], [28, 'male', 33.0, 3, 'no', 'southeast',
4449.462]]}
# ['age', 'sex', 'bmi', 'children', 'smoker', 'region', 'charges']
# age
# [[19, 'female', 27.9, 0, 'yes', 'southwest', 16884.924], [18, 'male', 33.77, 1,
'no', 'southeast', 1725.5523], [28, 'male', 33.0, 3, 'no', 'southeast', 4449.462]]
# [19, 'female', 27.9, 0, 'yes', 'southwest', 16884.924]
# 19

# 33.77
```

Records orientation:

```
      json_str = df.to_json(orient='records')
      df_json = json.loads(json_str)

      print(f'Records orientation:\n{df_json}')

      # How to drill down: list of dictionaries
      print(df_json[0]['age'])
      print(df_json[1]['bmi'])

      # Read json back into DataFrame using same format
      df_new = pd.DataFrame(pd.read_json(json_str, orient=&quot;records&quot;))


      # Output:
      # Records orientation:
      # [{'age': 19, 'sex': 'female', 'bmi': 27.9, 'children': 0, 'smoker': 'yes',
'region': 'southwest', 'charges': 16884.924}, {'age': 18, 'sex': 'male', 'bmi': 33.77,
'children': 1, 'smoker': 'no', 'region': 'southeast', 'charges': 1725.5523}, {'age': 28,
'sex': 'male', 'bmi': 33.0, 'children': 3, 'smoker': 'no', 'region': 'southeast',
'charges': 4449.462}]
      # 19

      # 33.77
```

Index orientation:

```
      json_str = df.to_json(orient='index')
      df_json = json.loads(json_str)

      print(f'Index orientation:\n{df_json}')

      # How to drill down: dictionary of dictionaries
      print(df_json['0']['age'])
      print(df_json['1']['bmi'])

      # Read json back into DataFrame using same format
      df_new = pd.DataFrame(pd.read_json(json_str, orient=&quot;index&quot;))


      # Output:
      # Index orientation:
      # {'0': {'age': 19, 'sex': 'female', 'bmi': 27.9, 'children': 0, 'smoker': 'yes',
'region': 'southwest', 'charges': 16884.924}, '1': {'age': 18, 'sex': 'male', 'bmi':
33.77, 'children': 1, 'smoker': 'no', 'region': 'southeast', 'charges': 1725.5523}, '2':
{'age': 28, 'sex': 'male', 'bmi': 33.0, 'children': 3, 'smoker': 'no', 'region':
'southeast', 'charges': 4449.462}}
      # 19
```

```
# 33.77
```

Values orientation:

```
json_str = df.to_json(orient='values')
df_json = json.loads(json_str)

print(f'Values orientation:\n{df_json}')

# How to drill down: list of lists
print(df_json[0][0])
print(df_json[1][2])

# Read json back into DataFrame using same format
df_new = pd.DataFrame(pd.read_json(json_str, orient=&quot;values&quot;))


# Output
# Values orientation:
# [[19, 'female', 27.9, 0, 'yes', 'southwest', 16884.924], [18, 'male', 33.77, 1,
'no', 'southeast', 1725.5523], [28, 'male', 33.0, 3, 'no', 'southeast', 4449.462]]
# 19

# 33.77
```

Table orientation:

```
json_str = df.to_json(orient='table')
df_json = json.loads(json_str)

print(f'Table orientation:\n{df_json}')

# How to drill down: dictionary of dictionaries of a list of dictionaries
print(df_json['schema']['fields'][1]['name'])
print(df_json['schema']['fields'][1]['type'])
print(df_json['data'][0]['age'])
print(df_json['data'][1]['bmi'])

# Read json back into DataFrame using same format
df_new = pd.DataFrame(pd.read_json(json_str, orient=&quot;table&quot;))


# Output:
# Table orientation:
# {'schema': {'fields': [{'name': 'index', 'type': 'integer'}, {'name': 'age',
'type': 'integer'}, {'name': 'sex', 'type': 'string'}, {'name': 'bmi', 'type': 'number'},
{'name': 'children', 'type': 'integer'}, {'name': 'smoker', 'type': 'string'}, {'name':
'region', 'type': 'string'}, {'name': 'charges', 'type': 'number'}], 'primaryKey':
['index'], 'pandas_version': '0.20.0'}, 'data': [{'index': 0, 'age': 19, 'sex': 'female',
'bmi': 27.9, 'children': 0, 'smoker': 'yes', 'region': 'southwest', 'charges': 16884.924},
```

```
{'index': 1, 'age': 18, 'sex': 'male', 'bmi': 33.77, 'children': 1, 'smoker': 'no',
'region': 'southeast', 'charges': 1725.5523}, {'index': 2, 'age': 28, 'sex': 'male',
'bmi': 33.0, 'children': 3, 'smoker': 'no', 'region': 'southeast', 'charges': 4449.462}]}
      # age
      # integer
      # 19

      # 33.77
```

Related Articles

- Native Python documentation
- Short and quick tutorial
- Detailed tutorial
- JSON with web service example
- JSON into DataFrame example

4.5SQLite3: Local SQL DB File

This video can be viewed online.

CSV to SQLite: Insurance Example

Writing to SQLite

Here is an example using the insurance database. Begin by reading the data into a DataFrame from the CSV file:

```
df = pd.read_csv('/content/drive/My Drive/Colab Notebooks/data/insurance.csv')
df


# Output
# [see in your own .ipynb file]
```

The next step is creating a new SQLite database. The connect() method serves double duty: it connects to an existing sqlite database, or creates one if none exists. The command below creates a database called 'insurance.db' since none exists yet. Later, the same command will open the one we just created.

```
conn = sqlite3.connect('insurance.db')
```

Since the database is empty, the first step is writing the SQL to create a table:

```
create_sql = &quot;CREATE TABLE IF NOT EXISTS customers (age INTEGER, sex TEXT,
bmi REAL, children INTEGER, smoker TEXT, region TEXT, charges REAL)&quot;
cursor = conn.cursor()
cursor.execute(create_sql)

# Results:
# &lt;sqlite3.Cursor at 0x7f1f2af445e0&gt;
```

For practice, here is how to insert the data row by row:

```
for row in df.itertuples():
  insert_sql = f&quot;INSERT INTO customers (age, sex, bmi, children, smoker,
region, charges) VALUES ({row[1]}, '{row[2]}', {row[3]}, {row[4]}, '{row[5]}', '{row[6]}',
{row[7]})&quot;
  cursor.execute(insert_sql)
```

If you examine the location where insurance.db is stored (you may need to refresh), you’ll notice another file called 'insurance.db-journal.' This is where

all of the newly inserted data exists. But it has not been committed to the original database yet. When using individual INSERT, UPDATE, and DELETE statements, we have to explicitly commit those changes, whereas the to_sql method we used previously implies a commit automatically.

```
conn.commit()
```

After running the commit method, you should see the '-journal' file disappear. But of course, we could always just commit the entire DataFrame all at once and avoid the row-by-row iteration:

```
df.to_sql(name=&quot;customers&quot;, con=conn, if_exists='replace', index=False)
# or use if_exists='append' to add data without deleting the original table
```

Reading our New SQLite DB

Now that we have a shiny new SQLite database, we can read from it. This time, the .connect() command will be used to open a connection to an existing database:

```
conn = sqlite3.connect('insurance.db')
```

As you can see, there is no output from creating a connection. But next, we will create SQL code to read a query from insurance.db.

```
        read_sql = &quot;SELECT * FROM customers&quot;
        cursor = conn.cursor()  # We already created the cursor above, but we would need
to create it if we hadn't
        results = cursor.execute(read_sql)

        print(results.fetchone(), '\n')
        results.fetchall()


        # Output:
        # (19, 'female', 27.9, 0, 'yes', 'southwest', 16884.924)
        # [(18, 'male', 33.77, 1, 'no', 'southeast', 1725.5523),
        #  (28, 'male', 33.0, 3, 'no', 'southeast', 4449.462),
        #  (33, 'male', 22.705, 0, 'no', 'northwest', 21984.47061),
        #  (32, 'male', 28.88, 0, 'no', 'northwest', 3866.8552),
        #  (31, 'female', 25.74, 0, 'no', 'southeast', 3756.6216),
        #  (46, 'female', 33.44, 1, 'no', 'southeast', 8240.5896),
        # ...
```

In the code above, we created a cursor() object as before. Then we execute some query on the cursor object. The results object has a variety of options for returning records from the query. The example includes two of the most common for "fetching" one record or all records.

The Pandas package also has built in functions for interacting with SQLite databases. The example below uses the DataFrame method read_sql_query() to return query results directly into a DataFrame to save you some time:

```
        df_new = pd.read_sql_query(read_sql, conn)
        df_new.head()
```

	age	sex	bmi	children	smoker	region	charges
0	19	female	27.900	0	yes	southwest	16884.92400
1	18	male	33.770	1	no	southeast	1725.55230
2	28	male	33.000	3	no	southeast	4449.46200
3	33	male	22.705	0	no	northwest	21984.47061
4	32	male	28.880	0	no	northwest	3866.85520

4.6Optional: Live Online SQL Databases

Optional Section

This section covers how to read data from live SQL-based relational database; Microsoft SQL Server in particular. This is just one of many examples. However, doing so requires that you have login credentials to a database. Most students don't have that. In that case, you may choose to skip this section. It will not be required for the assignment.

Beyond CSV files, Python can also read from and write to SQL-based relational databases. Doing so requires at least a cursory understanding of structured query language (SQL). Although this chapter will not cover SQL extensively, it is a very easy language to learn, at least well enough to read and write data. Two packages provide access to SQL Server, a Microsoft product used for small to large databases.

First, you may need to install the two packages:

```
!pip install pyodbc
!pip install pymssql
```

These two packages work a bit differently. The package pyodbc is generally preferred because it allows multithreading, makes it easier to convert the query results into a Pandas DataFrame, and works with both Azure- and non-Azure-based SQL Server instances. (Note: Azure is Microsoft's cloud server platform, similar to Amazon Web Services [AWS]). However, it is very difficult to install this package properly on Google Colab. Therefore, at least until Google Colab improves pyodbc support, I'd recommend only connecting to a non-Azure-based SQL Server instance using pymssql. Although pymssql is less preferred, the first example uses that package:

```
import _mssql

# Using dummy data
server = &quot;subdomain.domain.com&quot;
user = &quot;dbusername&quot;
pw = &quot;dbusernames_password&quot;
db = &quot;db_name&quot;

# Create a database connection object
conn = _mssql.connect(server=server, user=user, password=pw, database=db)

# Run the execute_query() method on that object. This includes the SQL necessary
# to pull out the records from the tables you are interested in. If you want to see
# the actual data this query is pulling from, you can download the dataset here:
# /content/drive/My Drive/Colab Notebooks/data/AmazonLawnAndGardenReviews.csv
conn.execute_query('SELECT TOP(5) * FROM amazon_products ORDER BY recordID')

# After query execution, the conn object contains a list of records that can be
iterated through:
for row in conn:
  # User the format row['fieldname'] to refer to the value in the row/column cell of
each record
  print(&quot;recordID={}, reviewText={}&quot;.format(row['recordID'],
row['reviewText']))


# Output
# recordID=1, reviewText=I like the product the sensor works well the minute it
comes in contact with a subject that crosses its path (that would be me for now) but it s
hard for me to tell if I am getting results. I live in a wooded area with lots of deer.
They come every day and more so at night  so I haven t seen any action yet.
# recordID=2, reviewText=My husband really likes this because he said it covers a
good amount of the lawn and he doesn t have to move it so often. It adjusts easily  its
sturdy and our grandkids had fun cooling off on those hot days too!
# recordID=3, reviewText=Needed to get another hummingbird feeder and this one is
perfect. It s almost like our other one  but is a little bigger and has four ports instead
of three. Now we have one in back of the house and one in front. PERFECT!!!!!  Well almost
perfect  we stillhave the mini battles  but at least there is another station for them to
fuel up.
```

```
      # recordID=4, reviewText=My husband just loves this sprinkler head. Works great
good distance and covers the area well. This is the second one he has purchased.
      # recordID=5, reviewText=Wonderful bird house!  Bought 2 and just waiting for some
tenants. Easy to install  well made. Have received complements for the neighbors. Very
happy with purchase.
```

For those who are interested and understand SQL, here are a few more examples of query formats you might run using pymssql:

```
      # examples of other query functions
      print(conn.execute_scalar(&quot;SELECT COUNT(*) FROM amazon_products&quot;))
      print(conn.execute_scalar(&quot;SELECT COUNT(*) FROM amazon_products WHERE
productName LIKE 'J%'&quot;))    # note that '%' is not a special character here
      print(conn.execute_row(&quot;SELECT * FROM amazon_products WHERE recordID=%d&quot;,
13))
      conn.execute_query('SELECT * FROM amazon_products WHERE productName=%s', 'Tomcat Rat
Snap Trap  1 Pack')

      for row in conn:
        print(row)

      conn.execute_query('SELECT * FROM amazon_products WHERE recordID IN (%s)', (5, 6))

      for row in conn:
        print(row)

      conn.execute_query('SELECT * FROM amazon_products WHERE productName LIKE %s',
'Tom%')

      for row in conn:
        print(row)

      conn.execute_query('SELECT * FROM amazon_products WHERE productName=%(productName)s
AND overall=%(overall)s', { 'productName': 'John Doe', 'overall': 4 } )

      for row in conn:
        print(row)

      conn.execute_query(&quot;SELECT * FROM amazon_products WHERE productName LIKE 'Tom%'
AND recordID IN (%s)&quot;, (10, 12, 13))

      for row in conn:
        print(row)
```

The pyodbc package is generally preferred because it works across both Azure- and non-Azure-based SQL Server instances. Please note that pyodbc may have driver issues when installing in the Google Colab environment. Although it’s possible to resolve these driver issues, it’s fairly complicated. Therefore, you may prefer to run this only on local (Anaconda/Jupyter)

.ipynb files or through Microsoft Notebooks (a cloud Python environment very similar to Google Colab):

```
import pyodbc
import pandas as pd

# Follow https://docs.microsoft.com/azure/sql-database/sql-database-connect-query-
Python to create a suitable database.
server = &quot;subdomain.domain.com&quot; # Change these values to your SQL Server
user = &quot;dbusername&quot;
pw = &quot;dbusernames_password&quot;
database = &quot;db_name&quot;
driver= '{ODBC Driver 13 for SQL Server}'
conn =
pyodbc.connect('DRIVER='+driver+';SERVER='+server+';PORT=1433;DATABASE='+database+';UID='+
user+';PWD='+ pw)

# If you want one row at a time:
cursor = conn.cursor()
sql = &quot;SELECT TOP(5) MaritalStatus, Gender, Income, Children, Cars, Age,
PurchaseBike FROM bb_BikeBuyers WHERE children &gt; 0&quot;
cursor.execute(sql)
row = cursor.fetchone()

while row:
  print(str(row[0]) + ',' + str(row[1]) + ',' + str(row[2]) + ',' + str(row[3]) +
',' + str(row[4]) + ',' + str(row[5]) + ',' + str(row[6]))
  row = cursor.fetchone()

print('\n')

# If you prefer to simply read the results in a Pandas DataFrame:
data = pd.read_sql(sql, conn)
data
```

```
Married,Male,60000,4,3,42,No
Married,Male,60000,3,2,66,No
Married,Male,80000,5,2,62,No
Married,Male,80000,5,1,47,Yes
Married,Male,40000,1,0,43,Yes
```

	MaritalStatus	Gender	Income	Children	Cars	Age	PurchasedBike
0	Married	Male	60000	4	3	42	No
1	Married	Male	60000	3	2	66	No
2	Married	Male	80000	5	2	62	No
3	Married	Male	80000	5	1	47	Yes
4	Married	Male	40000	1	0	43	Yes

For web applications, Python is most commonly used in association with (1) PostgreSQL and (2) MySQL databases. However, for data science projects, you may need to connect to many possible SQL databases, and there are different packages for each type of SQL database. Thankfully, all (or most) of these packages are built to follow the [PEP 249 Python Database API Specification v2.0](), which means that they will all basically work the same. If you learn these packages above, you should be in good shape to adapt to any other package for the database you need.

4.7Practice

To see how well you understand the concepts in this chapter, attempt the practice problems below:

Practice #1: Average Insurance Charge per Person

Using the insurance data provided in the prior sections (also available below):

- Read the CSV file into a Pandas DataFrame
- Cast the 'charges' column into a list
- Loop through the list and calculate an average charge score for all records

Example output:

```
13270.42
```

Practice #2: Feature Engineering AgeByBMI

You've spoken recently with a doctor about your insurance charge project. She told you that health typically declines as BMI increases, leading to greater insurance charges. However, she also said that effect increases with age. Therefore, you realized that a better prediction of insurance charges would be a new calculated field based on age * BMI. Complete the steps below:

- Read the CSV file into a Pandas DataFrame (if not already done)
- Create two lists by casting the ages and BMI values from the DataFrame into lists
- Create an empty list to store the new ageByBMI values called 'ageByBMI'
- Create a loop that will iterate through all age and bmi values; multiplying them to create a new ageByBMI score for each person and store it in the new empty list that was previously created
- Add the new list to the DataFrame immediately after the BMI column

- Print the first five values of the new DataFrame to confirm that the list was added properly

Example output:

```
   age     sex     bmi  children smoker     region      charges  ageByBMI

0   19  female  27.900         0    yes  southwest  16884.92400   530.100
1   18    male  33.770         1     no  southeast   1725.55230   607.860
2   28    male  33.000         3     no  southeast   4449.46200   924.000
3   33    male  22.705         0     no  northwest  21984.47061   749.265
4   32    male  28.880         0     no  northwest   3866.85520   924.160
```

Practice #3: Write to a File

Now that you've cleaned and prepared the dataset, it's time to share it with others who will perform the predictive modeling. Write your new dataset into a new CSV file called 'insurance_modified.csv'.

Example output:

```
# Look in your folder location and find the file you just created. Open it to make sure it looks right
```

4.8Assignment

Complete the assignment(s) below.

This assessment can be taken online.